



9. Decision Trees

As the name suggests, it uses a tree to decide an outcome

Previous Section:

 [8. Linear Regression](#)

Contents:

- [1. Define a Decision Tree](#)
- [2. Decision Tree follows the Recursive Splitting approach](#)
- [3. Decision Tree chooses the best feature for splitting using Information Gain](#)
- [4. Decision Trees with Python](#)
- [5. Decision Trees for Regression](#)

[Summary](#)

[References](#)

One of the most popular Machine Learning algorithms is a Decision Tree. And don't worry, you do *not* need to understand tree data structures to understand how a decision tree works.

Imagine you're playing a **Guess Who** game. You don't memorize all characters at once. Instead, you ask simple questions:

- *"Is this character female?"*
- *"Is this character a main character?"*
- *"Does this character have powers?"*

Each question **reduces the possibilities** until you reach the answer.

That's it. That's a Decision Tree.

1. Define a Decision Tree

A Decision Tree is a tree-based model that makes decisions **step by step**, using a sequence of simple questions.

Think of it like this:

- Each **question** → a decision
- Each **branch** → the outcome of that decision (Yes/No)
- Each **path** → a reasoning process
- Final result → the prediction

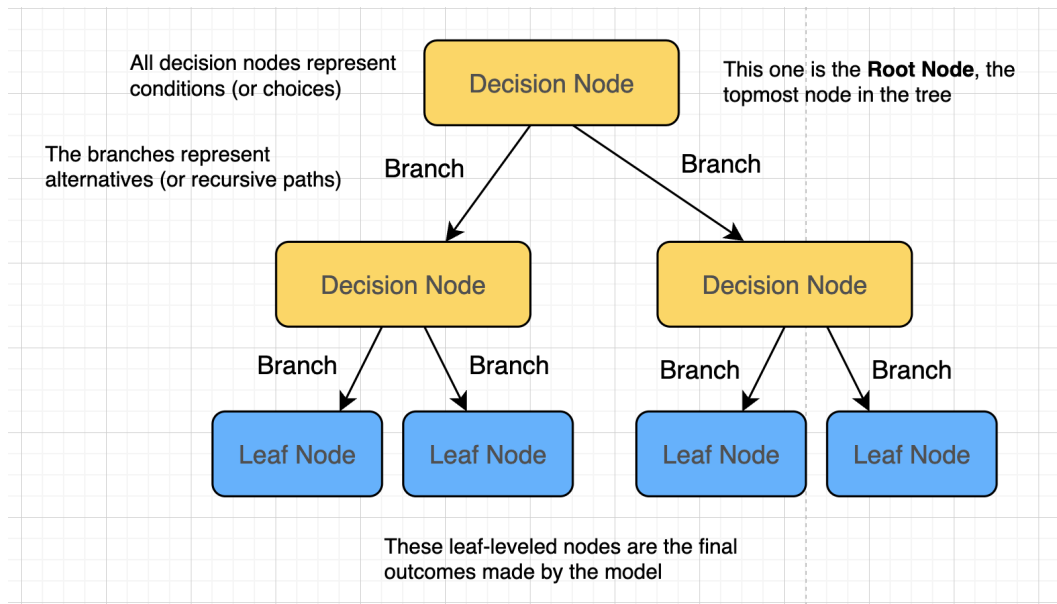
So instead of one complex formula, the model says:

┆ *"Let me break this problem into small, easy decisions."*

A Decision Tree really has **only three main components**:

- **Terminal Node (Leaf Node):** This is the **final point** in the tree. It represents the **decision made by the model**. There are **no further nodes after this** → Think of it as the **answer**.
- **Decision Node:** These are all the nodes **before reaching a terminal node**. Each node represents a **question**. The **root node** is the topmost decision node → This is where the model is still deciding what to do next.
- **Branches:** They are the **connections between nodes** → They represent the **outcome of a decision**
 - For binary cases → usually two branches (Yes/No)
 - For categorical features → can be more than two
 - For numerical features → values are often split into ranges (e.g., Age < 30, Age ≥ 30)

Reference the following image:



A Decision Tree is considered complete when: **Every branch leads to a terminal node**

It is also important to note that different branches can lead to the **same final decision**. But each path still represents a **unique reasoning process**.

There are two main types of Decision Trees, depending on the output:

- **Classification Tree (Decision Tree Classifier):** Used when the output is a **category (label)**. Examples:
 - Spam / Not Spam
 - Pass / Fail
 - Movie Genre Recommendation→ The output (the target class) is commonly referred to as the **Class Label**
- **Regression Tree (Decision Tree Regressor):** Used when the output is a **numerical value**. Examples:
 - House price
 - Temperature
 - Predicted rating→ The output in the regression problem is a **Continuous Value**

Consider an example “Should I Watch This Movie?”

The decision structure might look like this:

- Do you like Horror?
 - YES → Do you like Supernatural?
 - YES → Watch horror supernatural movie
 - NO → Watch realistic horror
 - NO → Do you like Romance?
 - YES → Watch romance
 - NO → Watch something else

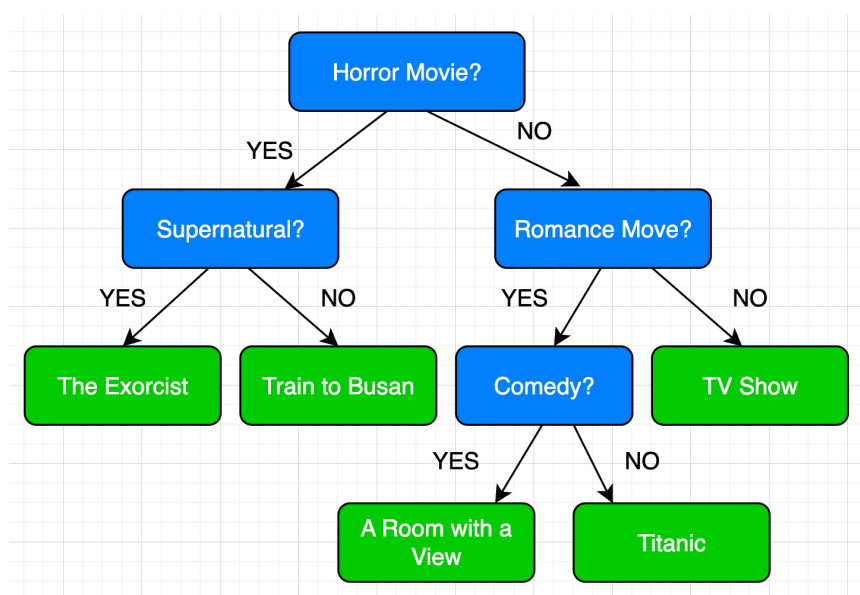
Notice something important:

At every step, there is **only ONE clear path forward**

No confusion. No overlap.

That’s why Decision Trees are easy to understand, easy to explain, and easy to trace. You can literally follow the logic like a story.

Now, if we actually **draw the Decision Tree** for our “Should I watch this movie?” problem, it would look like a branching structure:



Reading the tree:

- Start at the **root node**: *Do you like Horror?*
 - **Case 1: YES** → *Do you like Supernatural?*
 - YES → Watch a supernatural horror movie
 - NO → Watch a realistic horror movie
 - **Case 2: NO** → *Do you like Romance?*
 - YES → Watch a romance movie
 - NO → Watch something else (e.g., TV shows)

Each step follows:

- A **decision node** (question)
- A **branch** (answer)
- Ending at a **terminal node** (final recommendation)

2. Decision Tree follows the Recursive Splitting approach

The core idea behind Decision Trees is **Recursive Splitting**.

But more importantly: Recursive Splitting is the *reason* Decision Trees work so well.

A Decision Tree is built using a **top-down greedy approach**, and this naturally leads to a **Divide and Conquer strategy**. Instead of solving a complex problem all at once, the tree does something much simpler:

Break the problem into smaller pieces, solve each piece, and combine the results

This is exactly what we call **Divide and Conquer**.

Basically:

$$\begin{aligned}
 &(\text{Top-Down Induction} + \text{Greedy}) + \text{Recursion} \\
 &= \text{Divide and Conquer} + \text{Recursion} \\
 &= \text{Recursive Splitting}
 \end{aligned}$$

- **Top-down:** The process starts from the **root node**, where all training data is placed.
- **Greedy:** At each step, the algorithm chooses the **best possible split at that moment**, without considering future splits.

Here's how everything comes together:

1. Start with all data at the root
2. Choose the best split (**greedy choice**)
3. Divide the data into smaller groups (**divide**)
4. Repeat the same process on each group (**conquer**)

And this happens **again and again...**

That repetition is what we call **Recursion**, and that splitting is what we call **partitioning**

So Recursive Splitting is not just a step in the algorithm. **It is the algorithm!!**

The combination among:

- **Greedy** → always pick the best split now
- **Divide and Conquer** → break the problem into smaller parts
- **Recursive** → repeat the same process on each part

And together, they form the foundation of how a Decision Tree learns.

3. Decision Tree chooses the best feature for splitting using Information Gain

After introducing **Recursive Splitting**, we can now see how it actually plays out in practice.

- Once the first split is made:
 - The dataset is divided into **smaller subsets**
- From here, the tree simply continues the same idea:
 - Look at each subset
 - Ask a new question
 - Split again

And this process keeps repeating **within each branch**.

But notice something subtle here:

┆ The tree does not stop just because it made one split

It continues **only if the answer is still unclear**

- If a subset still contains both "Yes" and "No" → keep splitting
- If a subset contains only one outcome → stop

That stopping point becomes a **terminal node**

So over time, the tree is doing something very simple:

┆ Turning mixed groups into **pure groups**, step by step

And that is what gives us a clear and interpretable decision at the end.

Now we arrive at the most important question: **What is the "Best" Split?**

┆ If the tree keeps splitting... how does it decide **which feature to split on?**

Because at any point, there are multiple features to choose from. Each of them can split the data differently. So the goal of the model becomes:

┆ Choose the feature that **separates the data in the most meaningful way**

In other words:

- Reduce confusion as much as possible
- Make the resulting groups as "clean" as possible

There are several ways to measure this idea of a "good split". One of the most popular ways being the **Entropy and Information Gain**

And now, this naturally leads us to the next question:

┆ *How do we actually compute which split is the best?*

With that said, the premise of Decision Trees is to pick the best split. But the question is now **how?** Before jumping into formulas, let's think like humans:

┆ *What if we try to build the tree using intuition?*

Now, let's say we have this dataset, in which we have to predict whether a

person buys an XBOX:

Age	Income	Student	Credit_Rating	Buy_XBOX
≤30	high	no	Fair	no
≤30	high	no	Excellent	no
31...40	high	no	Fair	yes
>40	medium	no	Fair	yes
>40	low	yes	Fair	yes
>40	low	yes	Excellent	no
31...40	low	yes	Excellent	yes
≤30	medium	no	Fair	no
≤30	low	yes	Fair	yes
>40	medium	yes	Fair	yes
≤30	medium	yes	Excellent	yes
31...40	medium	no	Excellent	yes
31...40	high	yes	Fair	yes
>40	medium	no	Excellent	no

We try to choose AGE as the First Split:

- Age ≤ 30 → split further by INCOME?
 - High → XBOX? → Yes or No
 - Medium → XBOX? → Yes or No
 - Low → XBOX? → Yes or No
- Age: 31 ... 40 → XBOX? → Yes (already clear)
- Age > 40 → split further by INCOME?
 - High → (no data) → cannot determine
 - Medium → XBOX? → Yes or No
 - Low → XBOX? → Yes or No

Notice what happens:

- Some branches are still **mixed (Yes/No)**
- One branch is even **undefined (no data)**

→ This means the split is **not very good**

Now let's try another feature: **STUDENT**

- $\leq 30 \rightarrow$ STUDENT?
 - Yes $\rightarrow$ XBOX? $\rightarrow$ Yes or No
 - No $\rightarrow$ XBOX? $\rightarrow$ Yes or No
- $40 \rightarrow$ STUDENT?
 - Yes $\rightarrow$ XBOX? $\rightarrow$ Yes or No
 - No $\rightarrow$ XBOX? $\rightarrow$ Yes or No

Even here:

- We still have **mixed outcomes**
- We still need to **split further**

So what do we do? We keep applying **recursive splitting**

- Choose another feature
- Split again
- Repeat

Until each branch contains **only Yes or only No**

Now here's the important realization $\rightarrow$ **Not all splits are equally good**

- Some splits reduce uncertainty quickly
- Some splits leave a lot of confusion
- Some even create useless branches

The **order of splitting** has a huge impact on the accuracy of the model, the depth of the tree, and the efficiency of prediction

So the real challenge becomes: **How do we choose the best feature?**

We need a way to measure:

- How "clean" a split is
- How much uncertainty is reduced

- How informative a feature is

And this is where we introduce **Entropy and Information Gain**. Because instead of guessing like humans. The model will now use **mathematics** to decide the best split.

Now that we understand the tree needs to pick the **best split**, we need a way to measure: *How good is a split?* And this is where **Entropy** comes in.

Entropy measures **impurity** and **ambiguity** within a dataset, quantifying how much information, or lack of order is present. Essentially, it tells you **how messy and uncertain your data is**.

Think of it like this:

- If everything is mixed together → high entropy ("Your data is messy")
- If everything is clean and organized → low entropy ("Your data is well-organized")

Imagine a group of people:

- If half of them buy XBOX and half don't, it becomes very confusing to decide whether a given person buys XBOX or not → The classic 50/50 → This is the high-entropy case
- If almost everyone buys XBOX, it becomes very clear and somewhat "safe" to determine the given person as buys XBOX → Majority of them are → This is the low-entropy case
- If you find a specific feature—like "Subscribes to Game Pass"—that splits that messy 50/50 group into two perfectly organized groups (100% buy vs. 100% don't), you've just achieved **Information Gain**. You've used a rule to eliminate the messiness.

In a Decision Tree:

- The data is usually **mixed**. Different classes are all together → So the entropy is **high**

As the tree starts splitting:

- Each split tries to **separate the data**. Groups become more similar → Entropy starts to **decrease**

At a terminal node:

- Ideally, all samples belong to **one class only**. No more confusion and randomness → Entropy is at its **lowest**

Entropy tells us: "How uncertain is this group?"

- **High entropy** → very mixed → hard to decide
- **Low entropy** → very pure → easy to decide

Now that we understand entropy as "**how messy the data is**", let's briefly look at how we calculate it. For a single outcome:

$$Entropy(x) = -p(x) \log_2(p(x))$$

- $p(x)$ = probability of that outcome
- Entropy is always between 0 **and** 1

When there are only two classes (Yes / No):

$$Entropy(S) = -p \log_2(p) - (1 - p) \log_2(1 - p)$$

- p = probability of one class
- $1 - p$ = probability of the other

If there are more than two classes:

$$Entropy(S) = - \sum_{i=1}^k p_i \log_2(p_i)$$

Same idea: **Add up the uncertainty from all classes**

We now know how to measure **uncertainty**. But remember the goal of a Decision Tree is to **reduce uncertainty as much as possible**.

This is where **Information Gain (IG)** comes in.

Information Gain tells us how much **uncertainty is reduced after a split**

When we split the dataset:

- Each branch has its own entropy
- Some branches are more "pure" than others

So we compute a **weighted average entropy**:

$$Weighted\ Average = \sum p_k \cdot E_k$$

- p_k = proportion of data in each branch
- E_k = entropy of each branch

Now we compare:

- Before split → original entropy
- After split → weighted entropy

$$Information\ Gain(S, C) = E(S) - Weighted\ Average$$

- High Information Gain → big reduction in uncertainty
- Low Information Gain → little improvement

So at each step, the Decision Tree simply does this:

1. Try all possible features
 2. Compute Information Gain for each
 3. Pick the feature with the **highest Information Gain**
- That feature becomes the **next split (or root node)**

And that's the key: **The tree doesn't guess anymore... It uses Information Gain**

Applying Entropy & Information Gain (Step by Step)

Now let's apply everything we've learned.

We have **14 data points**, and the goal is to predict whether a person **buys an XBOX (Yes/No)**

- **Step 1 — Entropy of the Target**

First, we measure how "messy" the dataset is *before any split*.

- $P(Yes) = \frac{9}{14}$
- $P(No) = 1 - \frac{9}{14} = \frac{5}{14}$

So the entropy is:

$$Entropy(Target = XBOX) = -\frac{9}{4} * \log_2\left(\frac{9}{4}\right) - \frac{5}{4} * \log_2\left(\frac{5}{4}\right) \approx 0.9403$$

- The data is quite **mixed**
- Not completely random, but not pure either

→ So we definitely need to **split**

- **Step 2 — Entropy and Information**

Now we compute each feature and determine: How much does this feature **reduce entropy**?

Entropy and Information Gain for Each Attribute: We now compute which feature best predicts *Buy_XBOX*.

Attribute: Age

$$P(\text{Yes}) = \frac{2}{5}, \quad P(\text{No}) = \frac{3}{5}$$

$$Entropy(Age \leq 30) = -\frac{2}{5} \log_2\left(\frac{2}{5}\right) - \frac{3}{5} \log_2\left(\frac{3}{5}\right) = 0.9709$$

$$P(\text{Yes}) = 1, \quad P(\text{No}) = 0$$

$$Entropy(31 \leq Age \leq 40) = 0$$

$$P(\text{Yes}) = \frac{3}{5}, \quad P(\text{No}) = \frac{2}{5}$$

$$Entropy(Age > 40) = -\frac{3}{5} \log_2\left(\frac{3}{5}\right) - \frac{2}{5} \log_2\left(\frac{2}{5}\right) = 0.9709$$

$$\text{Weighted Average} = \frac{5}{14}(0.9709) + \frac{4}{14}(0) + \frac{5}{14}(0.9709) = 0.6935$$

$$IG(XBOX, Age) = 0.9403 - 0.6935 = 0.2468$$

Attribute: Student

$$P(\text{Yes}) = \frac{6}{7}, \quad P(\text{No}) = \frac{1}{7}$$

$$Entropy(Student = \text{Yes}) = -\frac{6}{7} \log_2\left(\frac{6}{7}\right) - \frac{1}{7} \log_2\left(\frac{1}{7}\right) = 0.5916$$

$$P(\text{Yes}) = \frac{3}{7}, \quad P(\text{No}) = \frac{4}{7}$$

$$Entropy(Student = \text{No}) = -\frac{3}{7} \log_2\left(\frac{3}{7}\right) - \frac{4}{7} \log_2\left(\frac{4}{7}\right) = 0.9852$$

$$\text{Weighted Average} = \frac{7}{14}(0.5916) + \frac{7}{14}(0.9852) = 0.7884$$

$$IG(XBOX, Student) = 0.9403 - 0.7884 = 0.1519$$

Attribute: Income

$$\text{High: } P(\text{Yes}) = \frac{2}{4}, \quad P(\text{No}) = \frac{2}{4}$$

$$\text{Entropy(High)} = -\frac{2}{4} \log_2 \left(\frac{2}{4} \right) - \frac{2}{4} \log_2 \left(\frac{2}{4} \right) = 1$$

$$\text{Medium: } P(\text{Yes}) = \frac{4}{6}, \quad P(\text{No}) = \frac{2}{6}$$

$$\text{Entropy(Medium)} = -\frac{4}{6} \log_2 \left(\frac{4}{6} \right) - \frac{2}{6} \log_2 \left(\frac{2}{6} \right) = 0.9182$$

$$\text{Low: } P(\text{Yes}) = \frac{3}{4}, \quad P(\text{No}) = \frac{1}{4}$$

$$\text{Entropy(Low)} = -\frac{3}{4} \log_2 \left(\frac{3}{4} \right) - \frac{1}{4} \log_2 \left(\frac{1}{4} \right) = 0.8112$$

$$\text{Weighted Average} = \frac{4}{14}(1) + \frac{6}{14}(0.9182) + \frac{4}{14}(0.8112) = 0.911$$

$$\text{IG}(XBOX, \text{Income}) = 0.9403 - 0.911 = 0.02929$$

Attribute: Credit Rating

$$\text{Fair: } P(\text{Yes}) = \frac{6}{8}, \quad P(\text{No}) = \frac{2}{8}$$

$$\text{Entropy(Fair)} = -\frac{6}{8} \log_2 \left(\frac{6}{8} \right) - \frac{2}{8} \log_2 \left(\frac{2}{8} \right) = 0.8112$$

$$\text{Excellent: } P(\text{Yes}) = \frac{3}{6}, \quad P(\text{No}) = \frac{3}{6}$$

$$\text{Entropy(Excellent)} = -\frac{3}{6} \log_2 \left(\frac{3}{6} \right) - \frac{3}{6} \log_2 \left(\frac{3}{6} \right) = 1$$

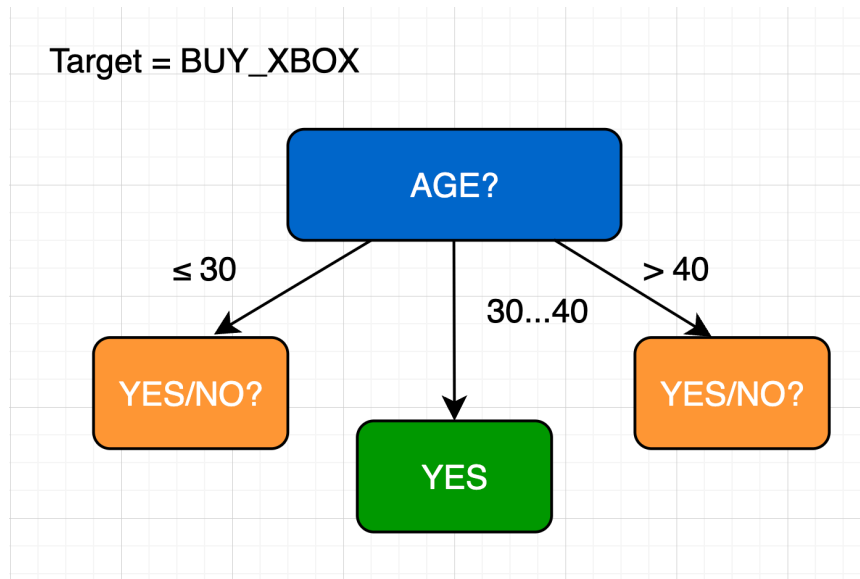
$$\text{Weighted Average} = \frac{8}{14}(0.8112) + \frac{6}{14}(1) = 0.8921$$

$$\text{IG}(XBOX, \text{CreditRating}) = 0.9403 - 0.8921 = 0.0482$$

The higher the Information Gain $\rightarrow$ the better the split.

$$\text{IG}(\text{Age}) = 0.2468 \quad (\text{Highest}) \Rightarrow \text{Root Node} = \text{Age}$$

Age becomes the **root node** since it has the highest Information Gain. Thus, it reduces uncertainty the most.



According to the dataset, when we split using **Age**, the resulting groups still contain **mixed labels**. This means that within the same Age group, there are samples where some people **buy the XBOX (Yes)** while others **do not (No)**.

In other words, even though the values of **Age are the same**, the outcomes are **different**.

→ This situation is called **impure data**.

A subset is considered **impure** when it contains **more than one class label** (both Yes and No). Because of this uncertainty, the model **cannot make a clear decision yet**, so it must **continue splitting** using other features.

Step 3 — A Naïve Approach (What NOT to Do)

Now you might think: ***"Why not just follow the ranking?"***

- First split on **Age** → Then **Student** → Then **Credit Rating** → Then **Income**

Seems like a logical approach. **But This is NOT How Decision Trees Work.**

This approach is **fixed and global**. It blindly follows the ranking, regardless of what happens after each split.

After splitting by **Age**, the dataset is already divided into smaller groups. Each group is **different now**. So the "best feature" might also be **different** in each branch.

This is where our earlier idea becomes important again: Decision Trees use **Recursive Splitting**

Instead of using the same feature order everywhere. We re-evaluate the best split **at each node**

- First split: choose best feature globally (Age)
- Next splits: choose best feature **locally** within each subset
 - The tree **adapts** as it grows

Information Gain is not used once. It is used **again and again at every step**. That's what makes Decision Trees flexible, adaptive, and much more powerful than a fixed rule system

Now we move to the **correct way** of building a Decision Tree. Instead of following a fixed order. We **recompute everything at each step**. This is the idea behind **Recursive Splitting** (used in algorithms like ID3).

At each node:

1. Take the **current subset of data**
2. Recompute **Entropy**
3. Recompute **Information Gain for all remaining features**
4. Choose the **best feature for THIS subset**

Not every subset (or node) needs to be split further. There are clear stopping conditions:

- If **all samples are labeled "Yes"**, we create a **leaf node = Yes**
- If **all samples are labeled "No"**, we create a **leaf node = No**

In both cases, the subset is considered **pure**, and we stop splitting.

Let's connect this to our dataset. Since we choose *Age* as our topmost feature. We will filter the dataset based on the unique classes in *Age*.

Sub-dataset 1: $Age \leq 30$

Age	Income	Student	Credit_Rating	Buy_XBOX
≤30	high	no	Fair	no
≤30	high	no	Excellent	no

Age	Income	Student	Credit_Rating	Buy_XBOX
≤30	medium	no	Fair	no
≤30	low	yes	Fair	yes
≤30	medium	yes	Excellent	yes

→ This subset is impure since it has mixed labels (*YES* and *NO*). So it requires more splitting

Sub-dataset 2: $Age = 31 \dots 40$

Age	Income	Student	Credit_Rating	Buy_XBOX
31...40	high	no	Fair	yes
31...40	low	yes	Excellent	yes
31...40	medium	no	Excellent	yes
31...40	high	yes	Fair	yes

→ This subset is **pure (all YES)**, so we stop here and assign Yes for every person with $Age = 31 \dots 40$

Sub-dataset 3: $Age > 40$

Age	Income	Student	Credit_Rating	Buy_XBOX
>40	medium	no	Fair	yes
>40	low	yes	Fair	yes
>40	low	yes	Excellent	no
>40	medium	yes	Fair	yes
>40	medium	no	Excellent	no

→ This subset is impure since it has mixed labels (*YES* and *NO*). So it requires more splitting

So we continue with the two impure subsets:

Sub-dataset: $Age \leq 30$

- **Step 1: Compute Entropy of the subset**

There are 5 samples: Yes = 2 and No = 3

$$Entropy = -\frac{2}{5} \log_2 \left(\frac{2}{5} \right) - \frac{3}{5} \log_2 \left(\frac{3}{5} \right) = 0.9709$$

Feature: Student

Student = Yes (2 samples): Yes = 2, No = 0 → Entropy = 0

Student = No (3 samples): Yes = 0, No = 3 → Entropy = 0

$$WeightedAverage = \frac{2}{5}(0) + \frac{3}{5}(0) = 0$$

$$IG(Age \leq 30 - > Student, XBOX) = 0.9709 - 0 = 0.9709$$

Feature: Income

High (2 samples): No, No → Entropy = 0

Medium (2 samples): Yes, No → Entropy = 1

Low (1 sample): Yes → Entropy = 0

$$WeightedAverage := \frac{2}{5}(0) + \frac{2}{5}(1) + \frac{1}{5}(0) = 0.4$$

$$IG(Age \leq 30 - > Income, XBOX) = 0.9709 - 0.4 = 0.5709$$

Feature: Credit Rating

Fair(3samples) : No, No, Yes → Entropy = 0.9182

Excellent(2samples) : No, Yes → Entropy = 1

$$WeightedAverage = \frac{3}{5}(0.9182) + \frac{2}{5}(1) = 0.9509$$

$$IG(Age \leq 30 - > CreditRating, XBOX) = 0.9709 - 0.9509 = 0.0200$$

$IG(Student) = 0.9709$ (highest) → So we split this subset based on the best feature **Student**

Sub-dataset: Age > 40

There are 5 samples: Yes = 3 and No = 2 → Entropy = 0.9709

Feature: Student

Student = Yes (3 samples): Yes, Yes, No → Entropy = 0.9182

Student = No (2 samples): Yes, No → Entropy = 1

$$WeightedAverage = \frac{3}{5}(0.9182) + \frac{2}{5}(1) = 0.9509$$

$$IG(Age > 40 - > Student, XBOX) = 0.9709 - 0.9509 = 0.0200$$

Feature: Income

Medium (3 samples): Yes, Yes, No → Entropy = 0.9182

Low (2 samples): Yes, No → Entropy = 1

$$WeightedAverage = \frac{3}{5}(0.9182) + \frac{2}{5}(1) = 0.9509$$

$$IG(Age > 40 - > Income, XBOX) = 0.0200$$

Feature: Credit Rating

Fair (3 samples): Yes, Yes, Yes → Entropy = 0

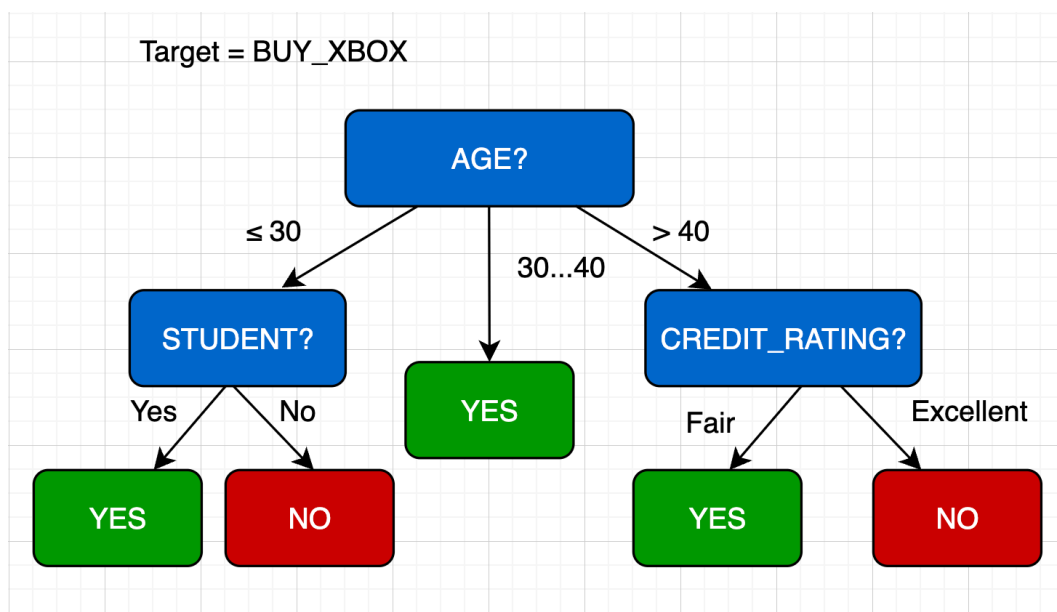
Excellent (2 samples): No, No → Entropy = 0

$$WeightedAverage = \frac{3}{5}(0) + \frac{2}{5}(0) = 0$$

$$IG(Age > 40 - > CreditRating, XBOX) = 0.9709$$

→ So we split this subset based on the **best feature Credit Rating**

Now we split again:



Sub-dataset: Age ≤ 30 & Student = YES

Age	Income	Student	Credit_Rating	Buy_XBOX
≤30	low	yes	Fair	yes

≤30	medium	yes	Excellent	yes
-----	--------	-----	-----------	-----

Sub-dataset: Age ≤ 30 & Student = NO

Age	Income	Student	Credit_Rating	Buy_XBOX
≤30	high	no	Fair	no
≤30	high	no	Excellent	no
≤30	medium	no	Fair	no

→ All of the sub-datasets are pure

Sub-dataset: Age > 40 & Credit-Ratings = Fair

Age	Income	Student	Credit_Rating	Buy_XBOX
>40	medium	no	Fair	yes
>40	low	yes	Fair	yes
>40	medium	yes	Fair	yes

Sub-dataset: Age > 40 & Credit-Ratings = Excellent

Age	Income	Student	Credit_Rating	Buy_XBOX
>40	low	yes	Excellent	no
>40	medium	no	Excellent	no

→ All of the sub-datasets are pure

At this point, all branches are **pure**, so the tree is complete. This is the essence of **Recursive Splitting**:

- At each step, we **only focus on the current subset**
- We **recompute everything locally**
- And we **stop as soon as the data becomes pure**

4. Decision Trees with Python

Here's how you would build a Decision Tree in Python using the Scikit-Learn library:

```

import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.preprocessing import OrdinalEncoder

# Define the dataset
data = {
    'Age': ['≤30', '≤30', '31...40', '>40', '>40', '>40', '31...40', '≤30', '≤30', '>40', '≤30', '31...40', '31...40', '>40'],
    'Income': ['high', 'high', 'high', 'medium', 'low', 'low', 'low', 'low', 'medium', 'low', 'medium', 'medium', 'medium', 'high', 'medium'],
    'Student': ['no', 'no', 'no', 'no', 'yes', 'yes', 'yes', 'no', 'yes', 'yes', 'yes', 'yes', 'no', 'yes', 'no'],
    'Credit_Rating': ['Fair', 'Excellent', 'Fair', 'Fair', 'Fair', 'Fair', 'Excellent', 'Excellent', 'Fair', 'Fair', 'Fair', 'Fair', 'Fair', 'Excellent', 'Excellent'],
    'Buy_XBOX': ['no', 'no', 'yes', 'yes', 'yes', 'no', 'yes', 'no', 'yes', 'yes', 'yes', 'yes', 'yes', 'yes', 'no']
}

# Create DataFrame
df = pd.DataFrame(data)

# Split features and target BEFORE encoding
target = 'Buy_XBOX'
X = df.drop(target, axis=1)
y = df[target]

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
                                                    random_state=42)

# Apply Ordinal Encoding

```

```

encoder = OrdinalEncoder()

X_train_encoded = encoder.fit_transform(X_train)
X_test_encoded = encoder.transform(X_test)

# Train the model
model = DecisionTreeClassifier(criterion="entropy", max_depth=3, random_state=0)
model.fit(X_train_encoded, y_train)

# Optional: Evaluate
accuracy = model.score(X_test_encoded, y_test)
print("Accuracy:", accuracy)

```

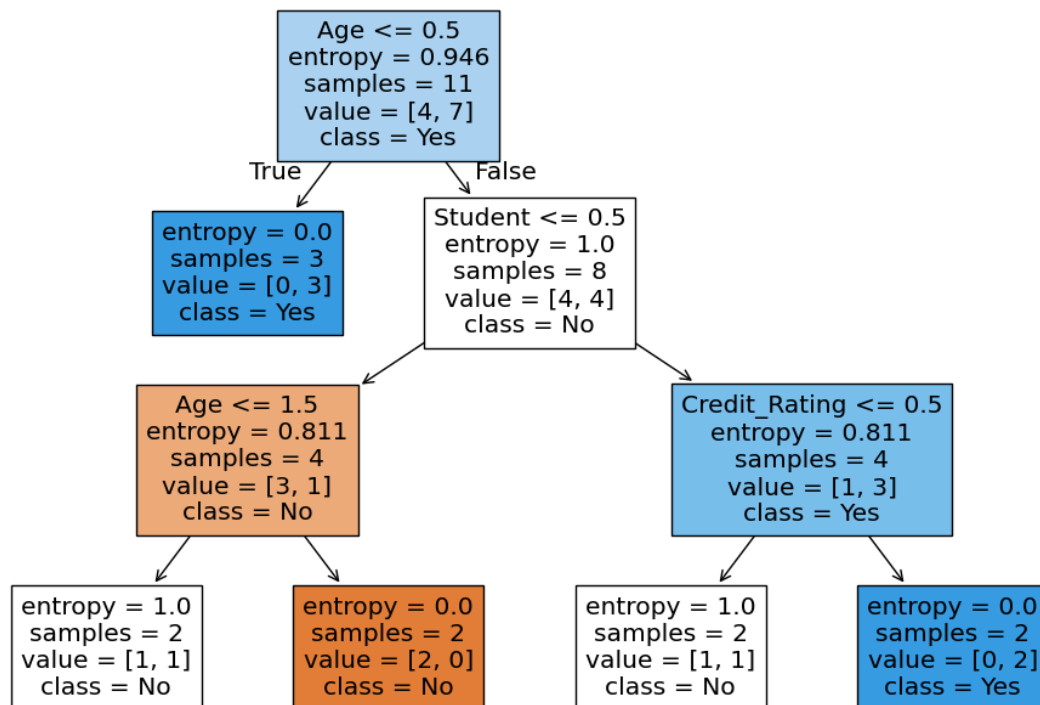
Accuracy: 1.0

```

# Draw the decision tree
from sklearn.tree import plot_tree
import matplotlib.pyplot as plt

plt.figure(figsize=(12, 8))
plot_tree(model, feature_names=X.columns, class_names=['No', 'Yes'], filled=True)
plt.show()

```



Other Splitting Methods

Besides Information Gain, there are other methods:

- **Gini Index** → measures impurity differently (used in CART)
- **Gain Ratio** → improves Information Gain by reducing bias

5. Decision Trees for Regression

Now, let's briefly look at how Decision Trees work for **Regression**.

The overall idea is **exactly the same** as in classification:

- We still **split the data recursively**
- We still try to **choose the best feature at each step**
- We still aim to make each subset as "pure" as possible

In **classification**, we measure impurity using: **Entropy** → **Information Gain (IG)**

But in **regression**, we are not predicting categories like *Yes/No*. We are predicting **continuous values** (e.g., price, salary, temperature).

So instead, we use **Standard Deviation Reduction (SDR)**

- Standard deviation measures **how spread out the values are**
- A good split is one that makes the values **more similar (less spread out)**

So the goal becomes:

→ **Reduce the variance as much as possible after the split**

$$SDR = SD(parent) - \sum \left(\frac{|subset|}{|parent|} \times SD(subset) \right)$$

- High SDR → good split (data becomes more consistent)
- Low SDR → bad split

The stopping condition is really similar to the like of classification:

- If the values in a subset are **very similar** → stop splitting
- The leaf node will output the **average value** of that subset

Classification Tree → reduce **uncertainty (Entropy)**

Regression Tree → reduce **spread (Standard Deviation)**

Same algorithm. Different measurement.

Summary

Let's summarize everything:

A decision tree makes decisions step by step using a top-down, greedy, divide-and-conquer approach known as recursive splitting, where the data is repeatedly partitioned based on the most informative features. To guide these splits, entropy is used to measure how disordered or "messy" the data is, while information gain determines which feature provides the best reduction in that disorder. At each node in the tree, these measures are recomputed so the model can adapt dynamically, choosing the most effective splits as it continues building the decision structure.

A Decision Tree is not just a model. It is a **structured way of thinking**. It asks a question, reduces uncertainty, and repeats until the answer becomes obvious.

And that is everything you need to know about **Decision Trees**.

References

<https://www.geeksforgeeks.org/machine-learning/decision-tree-introduction-example/>

<https://www.geeksforgeeks.org/machine-learning/decision-tree/>

<https://www.ibm.com/think/topics/decision-trees>

<https://www.datacamp.com/tutorial/decision-tree-classification-python>

<https://developers.google.com/machine-learning/decision-forests/decision-trees>

https://www.youtube.com/watch?v=_L39rN6gz7Y

<https://www.youtube.com/watch?v=LLBGiAAZqAM>

Next Section:

 [10. Random Forest](#)